YUKTII AI LABS
Algoridma Adhisthitam Ananta Jnana Patham
ENGINEERING INTELLIGENCE

Automated Car Painting Cell Simulation

Marah Ali · Mechatronics Engineering, Yildiz Technical University · October 2026

Introduction

Painting a car body is a repetitive task that needs consistent coverage, safe robot motion, and accurate positioning of the part. In this project, I built and programmed a simulated car-painting cell in RoboDK. A six-axis industrial robot paints the four sides of a car in turn while a turntable rotates the car to the correct angle, and the whole process runs automatically from a single main program.

The goals were:

- to build a station with a painting robot, a turntable and a car model;
- to program a paint path for each side of the car (0°, 90°, 180° and 270°);
- to make the robot return to a safe Home position between sides while the car rotates;
- to run the complete sequence automatically, without manually moving the turntable.

Station Setup

The station contains the following components:

- **Robot:** ABB IRB 6650S-90/3.9, mounted on an ABB IRBT 6004 linear track (rail), which lets the robot move along the car.
- **Tool:** a paint gun, defined as the robot's tool with its own tool centre point (TCP).
- **Turntable:** a rotating table (Turn Table, mounted on Turn Table Base) that carries the car. Four joint targets, turn_0, turn_90, turn_180 and turn_270, define its angles.
- **Car:** a 3D car model placed on the turntable, so that it rotates with it.
- **Home target:** a joint target that gives the robot a safe, clear position away from the car.

- **Reference frame:** a frame (Car_Frame) attached to the turntable, so the paint targets rotate together with the car.
- **Layout:** 11000x9000 block designed using SolidWorks.

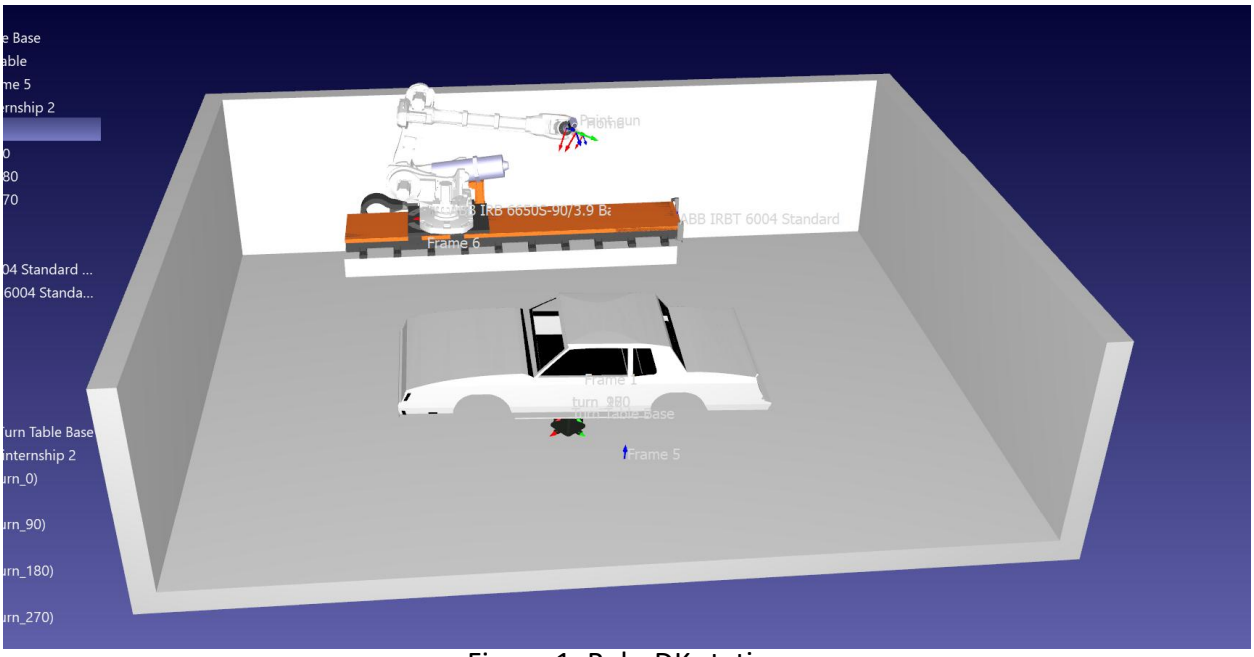


Figure 1: RoboDK station

Methodology

- Reference frame and tool:
  Paint targets are Cartesian positions, so they need a reference frame. I created a frame on the turntable and taught every paint target in it. Because the frame is attached to the turntable, the targets stay correct relative to the car whenever the table is at the matching angle. The paint gun was set as the active tool, so every target is reached with the gun's TCP.
- Paint paths for each side:

  1. An **approach point** a short distance in front of the first stroke, reached with a joint move (MoveJ).
  2. The **paint strokes**, reached with linear moves (MoveL) so the gun follows straight paths at a constant standoff from the surface.
  3. A **Home Position** is moved away from the surface to specify location.

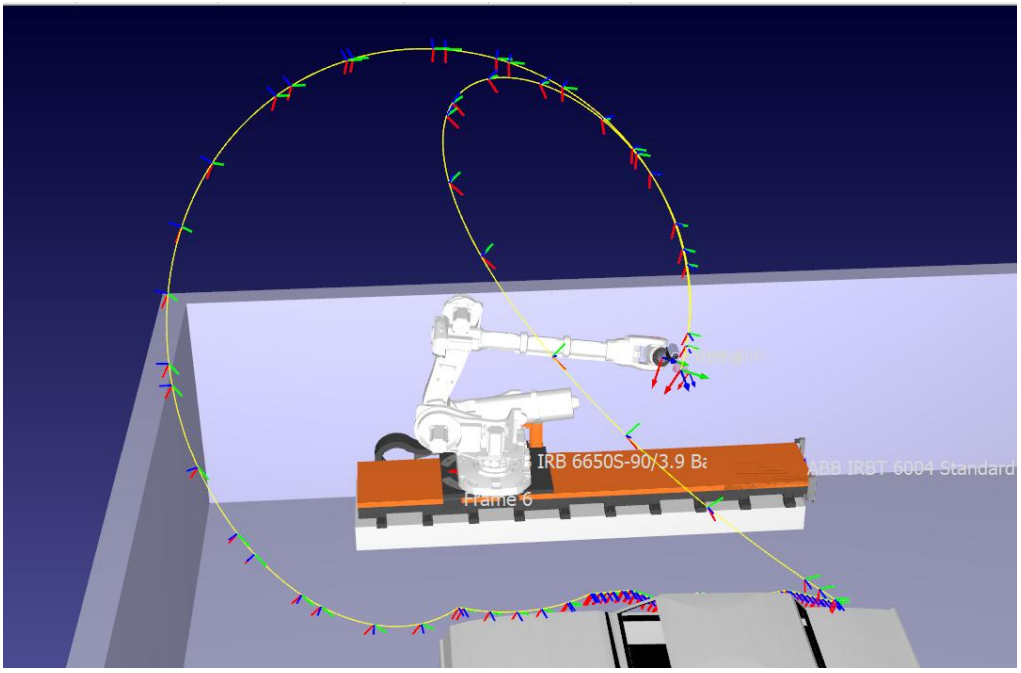


Figure 2: 0's position path

- Turntable programs:
  A program in RoboDK is linked to one device. When I put the turntable targets directly into the main program, the move was sent to the robot instead of the turntable. To fix this, I created one small program for each angle (`turn_0`, `turn_90`, `turn_180`, `turn_270`), each linked to the turntable and containing only the move to its target.
- Main program:
  The main program sets the reference frame and tool and then calls the other programs in order. The robot always returns to Home before the car rotates, so the arm is clear of the car while it turns:

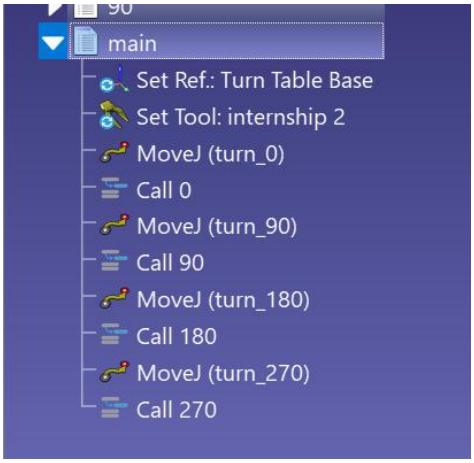


Figure 3: main program

Collision Test

I set up the collision map to watch the robot and gun against the car and the turntable, and left out pairs that touch by design (for example, the robot base on the rail). Running the programs with collision checking enabled showed which moves were unsafe. The approach and retreat points, and the return to Home between sides, keep the arm away from the car.

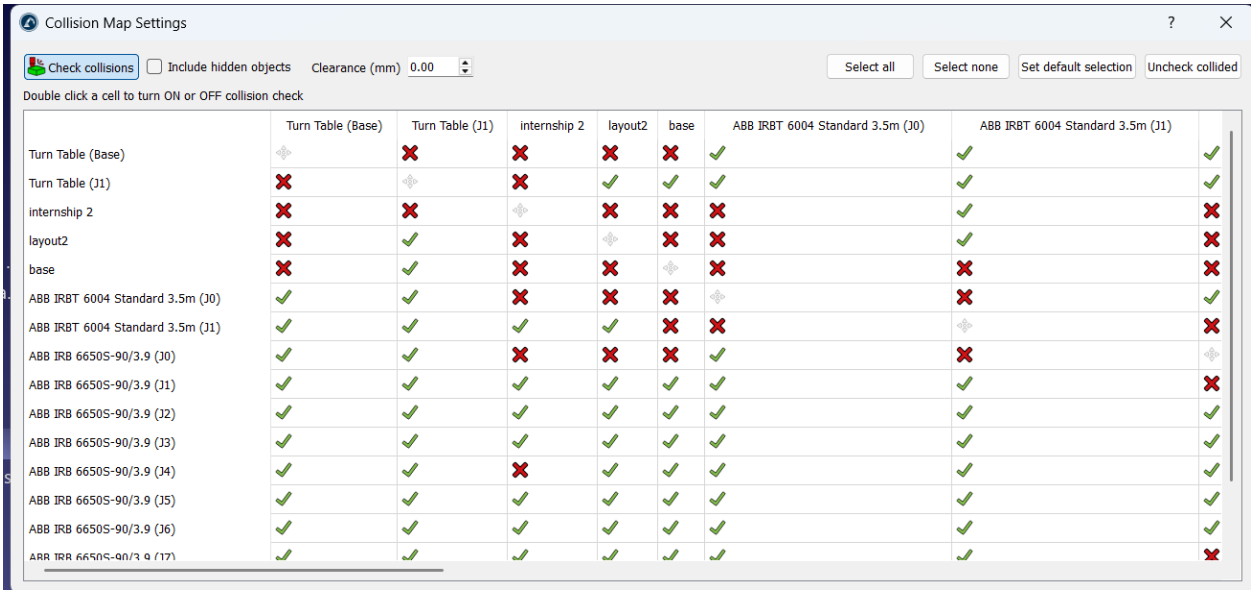


Figure 4: Collision Map

Results

The final program runs from start to finish without manual input. The robot paints the first side from the Home position and returns to Home; the turntable rotates the car by 90° and the robot paints the hood; the cycle repeats for the other side (180°) and the back (270°), and the robot ends at Home. No collisions or singularity errors occur during the run.

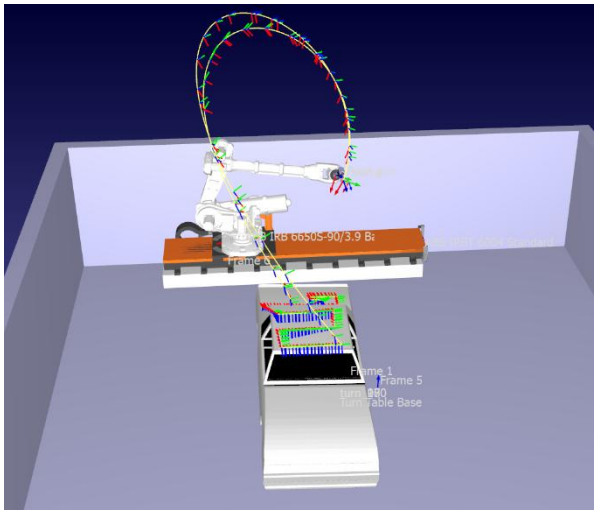


Figure 5: 90’s position path

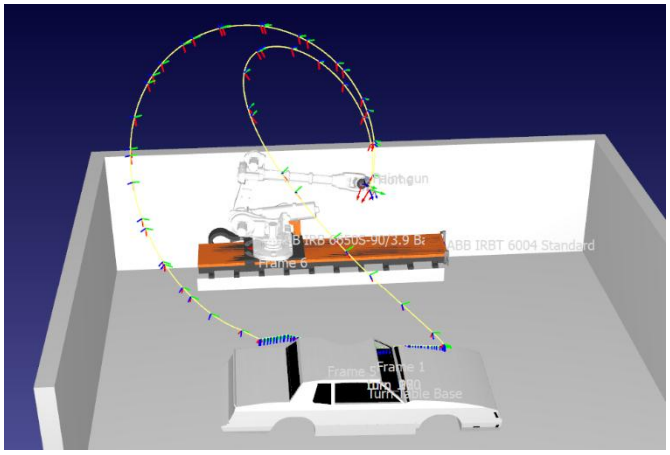


Figure 6: 180’s position path

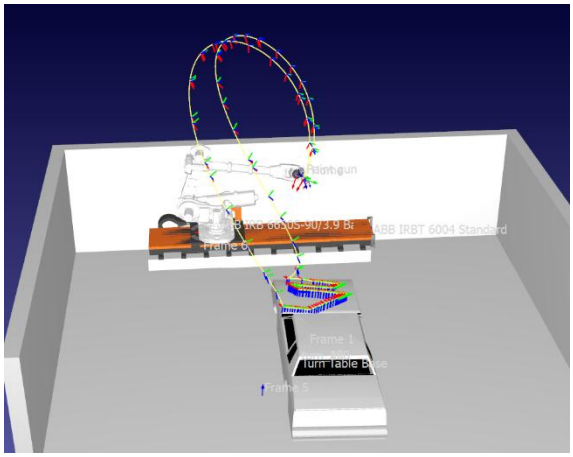


Figure 7: 270’s position path